AI Coder 缺陷场景测试集构建规划

2026-04-14 15:58

carsonchang

目录

AI Coder 缺陷场景测试集构建规划

1.2 缺陷来源

二、缺陷维度总览(10个维度 36个Case)

三、各维度详细设计

D1: 隐藏依赖发现失败

D2: 跨文件上下文丢失

D3: 代码幻觉

D4: 过时依赖

D5: 安全漏洞注入

D6: 调试衰减

D7: 规范偏离

D8: 测试覆盖不足

D9: 跨仓库/依赖迁移

D10: 长上下文遗忘

四、Case JSON 结构模板

五、Ground Truth 构建策略

5.1 自动提取(优先) - 适用D1/D2/D6/D9

5.2 手工构造 - 适用D3/D4/D5/D7/D8/D10

5.3 成本估算

六、候选仓库推荐

七、评测执行流程

八、与现有Benchmark的差异

九、语言级语义复杂性维度(D11-D20)

D11变量生命周期三态(4case)

D12全局变量副作用(3case)

D13回调函数与控制流反转(3case)

D14闭包与变量捕获(3case)

D15并发竞态条件(4case)

D16指针/引用别名(3case)

D17类型系统边界(3case)

D18异常传播路径(3case)

D19宏/模板/泛型展开(3case)

D20对象所有权与RAII(3case)

十、更新后总览

十一、高阶复杂场景D21-D34共36case

D21-D23 复合交织场景

D24-D26 跨编译阶段

D27-D29 隐式行为

D30-D32 状态机时序

D33-D34 内存模型

十二、完整总览D1-D34

按难度分层

按语言分布

carsonchang

AI Coder 缺陷场景测试集构建规划

文档正在编辑中...## 一、概述

构建一套精品测试集，系统性验证当前 AI Coder 的已知缺陷场景。每个缺陷维度 3-5 个 case，共计约 36 个，采用端到端功能验证方式。### 1.1 设计原则

原则	说明
缺陷驱动	每个case对应AI Coder的已知缺陷
端到端可执行	每个case必须有可运行的测试用例作为ground truth
多语言覆盖	C/C++ Python Go Java均覆盖
GT混合构建	能从git历史自动提取的自动，不能的手工补充
可复现	基于公开仓库构建

1.2 缺陷来源

- MIT 9大挑战(arXiv:2503.22625): 代码幻觉/过时依赖/上下文丢失/安全漏洞/测试不足/环境假设/规范偏离/调试衰减/协作障碍
- 2026年论文: ContextBench/CodeCompass/BeyondSWE/SWE-ContextBench/Codified Context

---

二、缺陷维度总览(10个维度 36个Case)

#	缺陷维度	AI Coder具体表现	Case数	来源
D1	隐藏依赖发现失败	无法找到无直接import关系但架构关键的文件	5	CodeCompass
D2	跨文件上下文丢失	修复bug时遗漏需同步修改的关联文件	5	ContextBench
D3	代码幻觉	生成调用不存在的API/包/函数	3	MIT C1
D4	过时依赖	调用已废弃API，不感知版本变迁	3	MIT C2
D5	安全漏洞注入	生成含SQL注入/XSS/路径遍历等漏洞代码	5	MIT C4
D6	调试衰减	多轮修复后陷入循环越改越错	3	MIT C8
D7	规范偏离	生成代码不符合项目编码规范	3	MIT C7
D8	测试覆盖不足	生成的测试覆盖率低遗漏边界	3	MIT C5
D9	跨仓库依赖迁移	无法处理涉及外部依赖升级的修改	3	BeyondSWE
D10	长上下文遗忘	长对话大文件中遗忘早期约束	3	SWE-ContextBench

三、各维度详细设计

D1: 隐藏依赖发现失败

缺陷表现: Agent用文本检索寻找相关代码，无法发现配置注入/反射/事件总线等间接关联文件。CodeCompass实验: 普通Agent 76.2% vs 图导航 99.4%。

验证目标: 给定bug，修复需要修改一个无直接import关系的文件，AI Coder能否发现？

Case	隐藏依赖类型	语言	场景
D1-01	配置注入/IoC	Java	Spring Bean注入的服务类修改需同步改XML/YAML配置
D1-02	事件总线/观察者	Python	EventEmitter的listener注册在另一模块，改事件签名需同步
D1-03	动态加载/反射	Go	字符串名称动态调用handler，重命名需同步注册表
D1-04	数据库schema关联	Python	ORM model字段修改需同步改migration和序列化器
D1-05	构建系统	C/C++	新增源文件未加入CMakeLists.txt致编译失败

GT构建: 从git历史筛选一个PR修改了多个无直接import关系文件的案例

评测: Pass=patch包含所有必要文件且测试通过; Fail=遗漏隐藏依赖文件

D2: 跨文件上下文丢失

缺陷表现: ContextBench发现Agent探索到的上下文与实际利用的之间存在巨大差距，修复不完整。

验证目标: 修复需同时修改3+个有直接依赖的文件，AI Coder能否全部正确修改？

Case	场景	语言	描述
D2-01	接口签名变更	Java	改接口方法签名需同步改所有实现类和调用方
D2-02	类型定义变更	Go	改struct字段需同步改序列化/反序列化/测试
D2-03	API契约变更	Python	改REST API返回格式需同步改handler/serializer/test/文档
D2-04	头文件变更	C/C++	改.h中函数签名需同步改所有.cc中的实现和调用
D2-05	多层继承修改	Python	改基类方法需验证所有子类override兼容性

GT构建: 从git历史筛选一个commit修改3+个有import关系文件的案例

评测: Pass=全部关联文件正确修改测试通过; Partial=部分正确; Fail=核心错误或遗漏关键文件

D3: 代码幻觉

缺陷表现: MIT研究38%代码含幻觉成分，调用不存在的API/包/错误签名。

Case	幻觉类型	语言	描述
D3-01	不存在的包	Python	使用冷门库实现功能，检查是否import了不存在的包
D3-02	错误API签名	Go	使用标准库API，检查参数顺序/类型是否正确
D3-03	混淆版本API	Java	使用某框架特定版本API，检查是否用了其他版本接口

GT构建: 人工构造，设计使用真实但冷门/易混淆API的任务

评测: Pass=编译通过+API真实存在+测试通过; Fail=ImportError或AttributeError

---

carsonchang

D4: 过时依赖

缺陷表现: 训练数据滞后，Agent调用已废弃API(如Python3.12中使用已移除的distutils)。

Case	过时类型	语言	描述
D4-01	已移除标准库模块	Python	Python3.12环境使用distutils/imp等已移除模块
D4-02	已废弃框架API	Java	SpringBoot3.x中使用2.x废弃注解
D4-03	已废弃语言特性	Go	使用Go1.22已废弃的ioutil包

GT构建: 人工构造，选已知deprecated API变迁点

评测: Pass=指定版本中API可用无deprecation warning; Fail=编译或运行失败

D5: 安全漏洞注入

缺陷表现: 12.1%的AI生成代码含安全漏洞(Asleep at the Keyboard)，高频XSS(18.5%)/SQL注入(15.2%)/路径遍历(12.8%)。

Case	漏洞类型(CWE)	语言	描述
D5-01	SQL注入(CWE-89)	Python	实现用户搜索接口，检查是否参数化查询
D5-02	XSS(CWE-79)	Java	实现评论展示页面，检查是否转义用户输入
D5-03	路径遍历(CWE-22)	Go	实现文件下载接口，检查是否验证路径
D5-04	命令注入(CWE-78)	Python	实现系统管理功能，检查shell命令构建安全性
D5-05	硬编码凭据(CWE-798)	Java	实现数据库连接，检查是否硬编码密码

GT构建: 人工构造+安全扫描。每case提供安全参考实现+注入攻击测试用例

评测: Pass=安全测试通过(注入被防御)+功能测试通过; Fail=注入攻击成功或CodeQL报漏洞

D6: 调试衰减

缺陷表现: Nature DDI模型:超6轮调试效率降至52%，倾向重复已失败方案。

Case	衰减场景	语言	描述
D6-01	误导性错误信息	Python	错误提示指向A文件但root cause在B文件
D6-02	多因素bug	C/C++	需同时修复两个独立问题才能通过测试
D6-03	回归引入	Go	修复bugA的方式会引入bugB，需更精巧方案

GT构建: 从git历史筛选一个Issue有多次修复尝试(多个相关commit)的案例

评测: Pass=6轮内修复测试通过; Partial=最终成功但超6轮; Fail=超10轮未修复或引入新bug

特殊指标: 轮次数/是否出现调试循环/修复方向是否收敛

D7: 规范偏离

缺陷表现: AGENTS.md研究发现即使提供规范文件Agent仍不遵守。MIT语义正确率仅61%。

Case	规范类型	语言	描述
D7-01	命名规范	Python	项目使用snake_case，检查是否混入camelCase
D7-02	错误处理风格	Go	项目要求错误wrap后返回，检查是否有裸return err
D7-03	架构约定	Java	项目严格分层Controller/Service/DAO，检查是否Controller直接写SQL

GT构建: 人工构造带CONTRIBUTING.md规范的项目+lint规则作为测试

评测: Pass=功能测试通过+lint检查通过; Fail=功能正确但违反规范

D8: 测试覆盖不足

缺陷表现: MIT研究AI生成代码平均测试覆盖率仅23%，边界条件常遗漏。

Case	测试场景	语言	描述
D8-01	边界条件遗漏	Python	为日期处理函数生成测试，检查闰年/月末/跨年覆盖
D8-02	异常路径遗漏	Go	为文件操作函数生成测试，检查权限不足/磁盘满/文件不存在
D8-03	并发场景遗漏	Java	为线程安全集合生成测试，检查多线程竞态条件

GT构建: 人工构造函数+预埋仅在边界条件触发的bug，正确测试应发现这些bug

评测: Pass=发现所有预埋bug+覆盖率>=80%; Partial=发现部分; Fail=未发现任何bug

D9: 跨仓库/依赖迁移

缺陷表现: BeyondSWE前沿模型跨仓库场景成功率低于45%。

Case	迁移类型	语言	描述
D9-01	主版本升级	Python	Flask 2.x->3.x迁移，API有breaking change
D9-02	依赖替换	Go	gorilla/mux迁移到chi(gorilla已归档)
D9-03	语言版本升级	Java	Java11->17，javax包改为jakarta

GT构建: 从真实仓库依赖升级PR中提取

评测: Pass=迁移后编译通过+测试通过+无deprecated warning; Fail=编译或测试失败

D10: 长上下文遗忘

缺陷表现: SWE-ContextBench错误上下文降低性能。MIT超10轮对话准确率降40%。

Case	遗忘场景	语言	描述
D10-01	大文件头部约束	Python	文件顶部type constraint注释，1000行后函数是否遵守
D10-02	早期对话约束	Go	对话开头指定error handling约定，10轮后是否记得
D10-03	多需求累积	Java	逐步提出5个需求约束，最终代码是否满足全部

GT构建: 人工构造大文件/长上下文中埋入约束，编写验证约束是否遵守的测试

评测: Pass=所有约束遵守测试通过; Partial=遗忘早期约束; Fail=核心约束被违反

四、Case JSON 结构模板

每个case统一结构: case_id dimension language severity issue(title+description+repo+base_commit) ground_truth(source+gold_patch+files_must_modify+test_patch) evaluation(method+test_commands+pass_criteria+fail_criteria) / metadata(paper_reference+difficulty+estimated_context_files)

详细模板见本地文件: docs/ai_coder_defect_testset_plan.md

五、Ground Truth 构建策略

5.1 自动提取(优先) - 适用D1/D2/D6/D9

SWE-bench Fail-to-Pass策略:

1. 爬取目标仓库PR列表
2. 按缺陷模式筛选: D1=修改无import关系多文件; D2=修改3+有import关系文件; D6=Issue多次commit; D9=标题含upgrade/migrate
3. 构建Docker环境
4. PR合入前运行测试->找到失败测试
5. PR合入后运行测试->确认通过

5.2 手工构造 - 适用D3/D4/D5/D7/D8/D10

1. 创建或fork目标仓库
2. 设计特定功能需求/Issue
3. 编写正确实现(gold patch)
4. 编写测试(正常+边界/安全/规范检查)
5. 验证: 不应用gold patch时测试失败，应用后通过

5.3 成本估算

方式	Case数	单Case工时	总工时
自动提取	~18(D1/D2/D6/D9)	2h	36h
手工构造	~18(D3/D4/D5/D7/D8/D10)	4h	72h
合计	~36		~108h(约14人天)

六、候选仓库推荐

语言	候选仓库	适合维度	理由
Python	Django	D1/D2/D5/D9	大型web框架，依赖复杂，SWE-bench已验证
Python	Flask	D4/D9	2.x->3.x有明确breaking change

carsonchang

语言	候选仓库	适合维度	理由
Python	scikit-learn	D2/D8	数值计算边界丰富
Go	kubernetes	D1/D2/D9	超大仓库模块间隐藏依赖多
Go	gin-gonic/gin	D5/D7	Web框架安全和规范场景丰富
Java	Spring Boot	D1/D4/D7/D9	IoC容器是隐藏依赖典型场景
Java	Apache Commons	D3/D4	API版本变迁丰富
C/C++	Redis	D2/D6	核心逻辑紧密耦合调试复杂
C/C++	curl	D5/D9	安全敏感依赖升级频繁

七、评测执行流程

1. 准备: checkout base_commit -> 构建Docker环境 -> 运行fail-to-pass测试确认失败
2. 执行: 将issue描述提供给AI Coder -> 收集Agent生成的patch -> 记录操作轨迹和轮次
3. 评测: 应用patch -> 运行fail-to-pass测试判定Pass/Fail -> 运行安全扫描(D5)/lint检查(D7)/覆盖率(D8)
4. 分析: 按维度统计Pass率 -> 对比不同AI Coder表现 -> 识别强项弱项

八、与现有Benchmark的差异

对比	SWE-bench	ContextBench	本测试集
评测重点	最终修复成功率	上下文检索过程	AI Coder特定缺陷暴露率
Case来源	自动PR提取	人工标注gold context	缺陷驱动设计+混合构建
语言覆盖	Python为主	8种语言	C/C++ Python Go Java
规模	2294	1136	~36(精品)
核心价值	能不能修	找没找对	哪里做不好
独特维度	无	无	代码幻觉/安全漏洞/调试衰减/规范偏离

完整文档含JSON模板详见本地: docs/ai_coder_defect_testset_plan.md

最后更新: 2026-04-14

九、语言级语义复杂性维度(D11-D20)

D1-D10聚焦工程能力缺陷(找不到文件/记不住上下文)。D11-D20聚焦对代码语义本身理解不深导致的错误——即使Agent找到了所有文件仍可能写出逻辑错误的代码。

#	缺陷维度	AI Coder具体表现	Case数	依据
D11	变量生命周期三态	不理解已初始化/未初始化/已释放的状态转换	4	LAMeD
D12	全局变量副作用	不理解全局变量被多处修改的连锁影响	3	Codified Context

#	缺陷维度	AI Coder具体表现	Case数	依据
D13	回调函数与控制流反转	不理解回调的异步执行顺序	3	ContextBench
D14	闭包与变量捕获	不理解闭包捕获变量时机(引用vs值)	3	RACE
D15	并发竞态条件	不理解共享资源竞态/错误同步原语	4	CONCUR(2026)
D16	指针/引用别名	不理解多个指针指向同一内存的影响	3	MIT C4
D17	类型系统边界	不理解隐式转换/整数溢出/浮点精度	3	RACE
D18	异常传播路径	不理解异常传播中资源泄漏/状态不一致	3	MIT C5
D19	宏/模板/泛型展开	不理解宏展开后的实际代码行为	3	CodeAgentBench
D20	对象所有权与RAII	不理解move语义/借用检查/析构顺序	3	LAMeD

D11变量生命周期三态(4case)

- D11-01 条件初始化: if分支初始化else未初始化却被使用(C/C++)
- D11-02 Use-after-free: 回调中引用已释放结构体(C/C++)
- D11-03 Double-free: 错误路径重复free(C)
- D11-04 悬垂指针: 返回局部变量引用(C++)
- 评测: 端到端(AddressSanitizer)+静态(Clang Analyzer)

D12全局变量副作用(3case)

- D12-01 全局配置联动: 改全局配置默认值需检查所有读取函数(C/C++)
- D12-02 全局状态机: 模块级变量控制状态机改转换逻辑影响多函数(Python)
- D12-03 全局注册表: 改全局map注册逻辑影响init链(Go)
- 评测: 端到端(所有依赖测试是否仍通过)

D13回调函数与控制流反转(3case)

- D13-01 异步回调时序: asyncio回调中访问已GC对象(Python)
- D13-02 事件回调重入: 回调中触发同一事件导致递归(C/C++)
- D13-03 回调参数生命周期: goroutine中回调引用已修改的外部切片(Go)
- 评测: 端到端(触发异步/重入场景的测试)

D14闭包与变量捕获(3case)

- D14-01 循环闭包陷阱: for循环启动goroutine闭包捕获循环变量(Go)
- D14-02 延迟求值: lambda在列表推导中捕获变量所有返回最后一个值(Python)
- D14-03 this/self捕获: lambda捕获外部类mutable字段并发修改不一致(Java)
- 评测: 端到端(设计期望不同值的测试)

D15并发竞态条件(4case)

- D15-01 缺少锁保护: 多goroutine并发写map未加mutex(Go)
- D15-02 死锁: 两线程按不同顺序获取两把锁(Java)

- D15-03 原子性违反: check-then-act非原子TOCTOU竞态(C++)
- D15-04 条件变量误用: Condition的wait/notify顺序错误(Python)
- 评测: 端到端(ThreadSanitizer)+压力测试
- 论文: CONCUR(arXiv:2603.03683, 2026)

D16指针/引用别名(3case)

- D16-01 参数别名: src和dst指向同一内存memcpy未检查overlap(C)
- D16-02 切片别名: 两个slice共享底层数组append导致覆盖(Go)
- D16-03 可变引用别名: list作为默认参数多次调用共享(Python)
- 评测: 端到端(触发别名问题的测试输入)

D17类型系统边界(3case)

- D17-01 整数溢出: int32乘法溢出未检查导致负数(C/C++)
- D17-02 浮点精度: 用==比较浮点数0.1+0.2!=0.3(Python)
- D17-03 隐式类型转换: int/long混用或Integer拆箱NPE(Java)
- 评测: 端到端(溢出/精度边界值测试)

D18异常传播路径(3case)

- D18-01 资源泄漏: try中打开文件异常时跳过close未用with(Python)
- D18-02 锁未释放: ReentrantLock手动lock后未在finally中unlock(Java)
- D18-03 状态不一致: defer中recover了panic但事务已部分提交未回滚(Go)
- 评测: 端到端(注入异常+资源泄漏检测)

D19宏/模板/泛型展开(3case)

- D19-01 宏多次求值: MAX宏对有副作用表达式i++多次求值(C)
- D19-02 模板特化陷阱: vector bool的意外特化行为(C++)
- D19-03 泛型类型擦除: 泛型擦除后运行时ClassCastException(Java)
- 评测: 端到端(触发宏副作用/类型擦除的测试)

D20对象所有权与RAII(3case)

- D20-01 Move后使用: std::move后继续访问obj成员(C++)
- D20-02 unique_ptr复制: 试图复制unique_ptr或用raw pointer绕过(C++)
- D20-03 Defer顺序: 多个defer的LIFO执行顺序被误解(Go)
- 评测: 端到端(编译检查+ASan/MSan)

十、更新后总览

类别	维度	Case数
工程能力缺陷	D1-D10	36
语言语义复杂性	D11-D20	32

类别	维度	Case数
总计	20个维度	68个case

总工时约200h(25人天)

完整内容见本地: docs/ai_coder_defect_testset_plan.md

最后更新: 2026-04-14

十一、高阶复杂场景D21-D34共36case

D11-D20覆盖单一语言特性。D21-D34覆盖多特性交织/跨编译阶段/隐式行为/状态机时序/硬件语义。

#	维度	表现	Case	分类
D21	并发+回调+生命周期交织	异步回调在另一线程执行引用已释放对象	3	复合交织
D22	闭包+并发+全局状态交织	多goroutine闭包捕获同一全局变量并发改	3	复合交织
D23	泛型+类型转换+异常链	downcast失败抛异常资源未清理	3	复合交织
D24	编译期vs运行期行为差异	混淆constexpr/const/runtime求值	3	跨阶段
D25	链接期符号冲突	ODR违反同名全局符号	2	跨阶段
D26	静态初始化顺序依赖SIOF	跨编译单元全局对象初始化顺序未定义	3	跨阶段
D27	隐式拷贝移动语义	编译器自动生成浅拷贝double-free	3	隐式行为
D28	运算符重载副作用	重载==有副作用在sort中被多次调用	2	隐式行为
D29	隐式类型提升链	char+char提升int与unsigned混合出错	3	隐式行为
D30	协议状态机违反	错误状态调用方法未握手就发数据	3	状态机
D31	重入与递归锁	持锁调用内部再获同锁死锁或破坏不变量	2	状态机
D32	信号处理异步安全	handler中malloc/printf非async-safe	2	状态机
D33	弱内存序	relaxed在x86正确ARM上失败	2	内存模型
D34	False sharing	相邻变量同cache line功能对性能差	2	内存模型

D21-D23 复合交织场景

- D21-01 定时器回调+worker线程+主线程已delete对象 Cpp
- D21-02 Channel回调+goroutine泄漏+defer未执行 Go
- D21-03 asyncio.Task cancel后await写已关闭连接 Python
- D22-01 多goroutine闭包捕获包级变量并发读写 Go
- D22-02 线程池lambda捕获共享vector并发push_back Cpp
- D22-03 多线程lambda访问未完全初始化单例 Java
- D23-01 List泛型擦除ClassCast未关闭DB连接 Java
- D23-02 模板dynamic_cast返回nullptr未检查解引用 Cpp

- D23-03 interface类型断言失败panic defer丢失错误 Go

D24-D26 跨编译阶段

- D24-01 constexpr处理运行时输入编译失败 Cpp
- D24-02 static_assert用运行时条件 Cpp
- D24-03 const iota枚举编译器优化 Go
- D25-01 两个cc文件同名inline函数实现不同 Cpp
- D25-02 两个c文件tentative definition全局变量 C
- D26-01 跨文件全局Registry依赖未初始化对象 Cpp
- D26-02 Meyers Singleton替代全局变量 Cpp
- D26-03 Go包init循环依赖 Go

D27-D29 隐式行为

- D27-01 含raw pointer类默认浅拷贝double-free Cpp
- D27-02 Python切片赋值浅拷贝vs深拷贝 Python
- D27-03 声明析构函数导致NRVO失败性能退化 Cpp
- D28-01 operator==写日志用作map key日志爆炸 Cpp
- D28-02 __lt__不满足传递性sort结果不确定 Python
- D29-01 unsigned len-1下溢为UINT_MAX C
- D29-02 signed char溢出UB C
- D29-03 uint16_t左移16位signed overflow UB Cpp

D30-D32 状态机时序

- D30-01 未begin就commit数据库事务 Python
- D30-02 Response.Body未Close复用连接泄漏 Go
- D30-03 连接池未归还后续获取阻塞 Java
- D31-01 持mutex调用用户回调回调又获取同锁 Cpp
- D31-02 ReentrantLock重入时数据不一致 Java
- D32-01 SIGTERM handler中malloc堆损坏 C
- D32-02 SIGUSR1 handler中fprintf stdout损坏 C

D33-D34 内存模型

- D33-01 relaxed原子flag+data另一线程读到旧值 Cpp
- D33-02 Go channel happens-before语义误解 Go
- D34-01 相邻atomic计数器false sharing Cpp
- D34-02 多goroutine操作数组相邻元素 Go

十二、完整总览D1-D34

类别	维度	Case数	工时
工程能力缺陷	D1-D10	36	108h(14人天)

类别	维度	Case数	工时
语言语义复杂性	D11-D20	32	92h(12人天)
高阶复杂场景	D21-D34	36	126h(16人天)
总计	34个维度	104个case	326h(约41人天)

按难度分层

层级	维度	Case数	优先级
L1基础	D1-D10	36	必做
L2进阶	D11-D20	32	推荐
L3高阶	D21-D29	22	按需
L4专家	D30-D34	14	选做

按语言分布

语言	Case数	主要维度
C/C++	约38	D11 D16 D19-D20 D24-D29 D31-D34
Python	约24	D2-D5 D8 D12-D14 D18 D28 D30
Go	约24	D1 D3 D6 D9 D13-D15 D22 D26 D30
Java	约18	D1 D5 D7 D14-D15 D17 D23 D28 D31

完整详情见本地: docs/ai_coder_defect_testset_plan.md

最后更新: 2026-04-14

carsonchang